INTERVIEW PREP SERIES

JavaScript

Interview Questions & Easy Answers

---

60 questions · 29 marked ★ most asked

Covers basics, scope and hoisting, closures, `this`, prototypes, async JS (callbacks, promises, async/await, event loop), ES6+ features, DOM and the output-based questions interviewers love.

How to use this document

- Questions go from basic to advanced inside each section. Freshers should master the ★ ones first.
- Answers are written in simple words, the way you would say them in an interview. Learn the idea, not the exact sentences.
- Questions are based on technical rounds reported by candidates on GeeksforGeeks, LeetCode Discuss, Glassdoor, Naukri and similar interview-experience pages, plus the standard question banks interviewers draw from.
- Read one section a day and say each answer out loud once. Speaking it is what makes it stick.

Basics

Q1. What is JavaScript and where does it run? ★ Most asked

JavaScript is a high-level, single-threaded, dynamically typed programming language. It was made for web pages, so it runs in every browser, but with **Node.js** it also runs on servers.

It is used to make pages interactive, build backends, mobile apps (React Native) and even desktop apps (Electron).

Q2. What are the data types in JavaScript? ★ Most asked

There are 8 types. Seven are **primitive**: `string`, `number`, `bigint`, `boolean`, `undefined`, `null`, `symbol`. The eighth is **object** (arrays, functions, dates are all objects).

Primitives are copied by value; objects are copied by reference.

Q3. What is the difference between `var`, `let` and `const`? ★ Most asked

- `var` is function-scoped, can be re-declared, and is hoisted with value `undefined`.
- `let` is block-scoped (inside `{ }`), cannot be re-declared in the same scope, and is in the "temporal dead zone" until its line runs.
- `const` is like `let` but cannot be re-assigned. The object it points to can still change.

```
const user = { name: "A" };
user.name = "B";   // allowed
user = {};         // TypeError
```

Interview tip: Say "I use const by default, let when the value changes, and avoid var."

Q4. What is the difference between == and ===? ★ Most asked

`==` compares after converting types (loose equality). `===` compares value AND type with no conversion (strict equality).

```
5 == "5"    // true
5 === "5"   // false
null == undefined   // true
null === undefined  // false
```

Always prefer `===` to avoid surprises.

Q5. What is the difference between `null` and `undefined`? ★ Most asked

`undefined` means a variable was declared but no value was given. `null` is a value you set on purpose to say "empty".

`typeof undefined` is "undefined" but `typeof null` is "object" (an old bug in JS).

Q6. What is `NaN`? How do you check for it?

`NaN` means "Not a Number", the result of a bad math operation like `"abc" * 2`. Strangely, `NaN === NaN` is false, so use `Number.isNaN(x)` to check.

Q7. What is type coercion?

Coercion is JavaScript automatically converting one type into another.

```
"5" + 2   // "52"  (number becomes string)
"5" - 2   // 3     (string becomes number)
true + 1  // 2
```

`+` with a string joins text; other math operators convert to numbers.

Q8. What are truthy and falsy values?

Falsy values become `false` in an if-condition. There are only a few: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. Everything else is truthy, including `"0"`, `[]` and `{}`.

Q9. What is the difference between primitive and reference types?

Primitives (number, string…) are stored directly and copied by value. Objects and arrays are stored as a reference (address), so copying the variable copies the address, and both variables point to the same object.

```
let a = [1]; let b = a; b.push(2);
console.log(a); // [1, 2]
```

Scope, hoisting and closures

Q10. What is hoisting? ★ Most asked

Hoisting means declarations are moved to the top of their scope before code runs.

- `var` is hoisted and set to `undefined`.
- `let`/`const` are hoisted but not initialised (accessing them early gives a ReferenceError).
- Function declarations are fully hoisted, so you can call them before you write them.

```
console.log(x); // undefined
var x = 5;
greet();        // works
function greet() { console.log("hi"); }
```

Q11. What is the Temporal Dead Zone (TDZ)?

The TDZ is the time between entering a scope and the line where a `let` or `const` variable is declared. Using the variable inside this zone throws a ReferenceError.

Q12. What is scope? What are its types? ★ Most asked

Scope decides where a variable can be accessed.

- **Global scope**: available everywhere.
- **Function scope**: available only inside that function.
- **Block scope**: available only inside `{ }` (for `let` and `const`).

JavaScript uses **lexical scope**, which means inner functions can use variables of outer functions based on where the code is written.

Q13. What is a closure? Give an example. ★ Most asked

A closure is when an inner function remembers the variables of its outer function, even after the outer function has finished running.

```
function counter() {
  let count = 0;
  return function () { count++; return count; };
}
const inc = counter();
inc(); // 1
inc(); // 2
```

Uses: data privacy, counters, memoization, and event handlers.

Interview tip: This is the #1 JS question in fresher interviews. Be ready to write this counter example.

Q14. What is the output of this loop and why? ★ Most asked

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
```

It prints `3 3 3`. `var` is function-scoped, so all three callbacks share the same `i`, which is 3 when they finally run. Changing `var` to `let` prints `0 1 2`, because `let` creates a new `i` for each loop iteration.

Q15. What is an IIFE?

An Immediately Invoked Function Expression is a function that runs as soon as it is defined. It was used to create a private scope before `let` existed.

```
(function () { console.log("runs now"); })();
```

Q16. What is the scope chain?

When JS looks for a variable, it first checks the current scope, then the outer scope, and so on up to the global scope. This lookup path is called the scope chain. If it is not found anywhere, you get a ReferenceError.

Functions

Q17. What is the difference between a normal function and an arrow function? ★ Most asked

- Arrow functions have no own `this`; they take `this` from the surrounding code.
- They have no `arguments` object.
- They cannot be used as constructors with `new`.
- They have shorter syntax: `const add = (a, b) => a + b;`

Q18. What is the difference between a function declaration and a function expression?

A declaration (`function f() {}`) is hoisted fully. An expression (`const f = function () {}`) is not usable before its line, because only the variable is hoisted.

Q19. What are first-class functions?

In JavaScript, functions are treated like any other value. You can store them in variables, pass them as arguments, and return them from other functions.

Q20. What is a higher-order function? ★ Most asked

A function that takes another function as an argument or returns a function. `map`, `filter`, `reduce`, `setTimeout` and `addEventListener` are all higher-order functions.

Q21. What is a callback function? ★ Most asked

A callback is a function passed into another function to be called later, often when some task finishes.

```
setTimeout(function () { console.log("done"); }, 1000);
```

Too many nested callbacks lead to "callback hell", which promises solve.

Q22. What is currying?

Currying converts a function with many arguments into a chain of functions taking one argument each.

```
const add = a => b => c => a + b + c;
add(1)(2)(3); // 6
```

Q23. What are default parameters and rest parameters?

Default parameters give a value if none is passed: `function hi(name = "Guest") {}`.

Rest parameters collect remaining arguments into an array: `function sum(...nums) { return nums.reduce((a, b) => a + b, 0); }`

Q24. What is the difference between the spread and rest operator? ★ Most asked

Both use `...`. **Spread** expands an array or object into individual items (`[...arr, 4]`, `{...obj}`). **Rest** collects many items into one array (in function parameters or destructuring).

Q25. What are pure functions?

A pure function always returns the same output for the same input and does not change anything outside itself (no side effects). Pure functions are easy to test and are central to React and Redux.

this, objects and prototypes

Q26. What is `this` in JavaScript? ★ Most asked

`this` refers to the object that is calling the function. Its value depends on how the function is called:

- In a method `obj.fn()`, `this` is `obj`.
- In a normal function call (non-strict mode), it is the global object; in strict mode it is `undefined`.
- In an arrow function, it is taken from the outer scope.
- With `new`, it is the newly created object.

Q27. What is the difference between `call`, `apply` and `bind`? ★ Most asked

All three set the value of `this`.

- `call(thisArg, a, b)` calls the function immediately with arguments one by one.
- `apply(thisArg, [a, b])` calls immediately with arguments as an array.
- `bind(thisArg)` returns a new function with `this` fixed, to be called later.

```
function hi(greet) { return greet + " " + this.name; }
const u = { name: "Riya" };
hi.call(u, "Hello");     // "Hello Riya"
hi.apply(u, ["Hey"]);    // "Hey Riya"
const f = hi.bind(u); f("Hi");
```

Q28. What is prototypal inheritance? ★ Most asked

Every JS object has a hidden link to another object called its **prototype**. If a property is not found on the object, JS looks at its prototype, then the prototype's prototype, and so on (the prototype chain). This is how objects inherit methods, e.g. every array gets `push` from `Array.prototype`.

Q29. What are classes in JavaScript?

Classes (ES6) are a cleaner syntax over prototypes.

```
class Animal {
  constructor(name) { this.name = name; }
```

```
  speak() { return this.name + " makes a sound"; }
}
class Dog extends Animal {
  speak() { return this.name + " barks"; }
}
```

Q30. What is the difference between shallow copy and deep copy? ★ Most asked

A shallow copy copies only the top level; nested objects are still shared. A deep copy copies everything.

```
const shallow = { ...obj };              // or Object.assign({}, obj)
const deep = structuredClone(obj);       // modern deep copy
const deep2 = JSON.parse(JSON.stringify(obj)); // older way, loses functions/dates
```

Q31. What is destructuring?

A short way to pull values out of arrays or objects into variables.

```
const { name, age } = user;
const [first, second] = [10, 20];
```

Q32. What is the difference between `Object.freeze` and `Object.seal`?

`freeze` makes an object fully read-only: no add, delete or change. `seal` stops adding or deleting properties but still allows changing existing ones. Both are shallow.

Q33. What is optional chaining and nullish coalescing?

Optional chaining `?.` stops and returns `undefined` if something is null/undefined, instead of crashing: `user?.address?.city`.

Nullish coalescing `??` gives a default only when the left side is `null` or `undefined`: `count ?? 0`. Unlike `||`, it keeps `0` and `""`.

Arrays and strings

Q34. What is the difference between `map`, `filter` and `reduce`? ★ Most asked

- `map` transforms every item and returns a new array of the same length.
- `filter` keeps only items that pass a condition.
- `reduce` combines all items into one value (sum, object, etc.).

```
const nums = [1, 2, 3, 4];
nums.map(n => n * 2);              // [2,4,6,8]
nums.filter(n => n % 2 === 0);     // [2,4]
nums.reduce((sum, n) => sum + n, 0); // 10
```

Q35. What is the difference between `forEach` and `map`?

`forEach` just runs a function on each item and returns `undefined`. `map` returns a new array. Use `map` when you need the result.

Q36. What is the difference between `slice` and `splice`?

`slice(start, end)` returns a copy of part of the array and does not change the original. `splice(start, count, ...items)` changes the original array by removing or adding items.

Q37. How do you remove duplicates from an array? ★ Most asked

```
const unique = [...new Set([1, 2, 2, 3, 3])]; // [1,2,3]
```

A `Set` only stores unique values, and spread turns it back into an array.

Q38. How do you reverse a string in JavaScript?

```
const rev = str.split("").reverse().join("");
```

Split into characters, reverse the array, then join back.

Q39. What is the difference between `for...in` and `for...of`?

`for...in` loops over the **keys** (property names) of an object. `for...of` loops over the **values** of an iterable like an array, string, Map or Set.

Q40. What are Map and Set?

`Map` stores key-value pairs where keys can be any type and order is kept. `Set` stores unique values. Both have `.size`, `.has()`, `.delete()` and are faster than objects for frequent add/remove.

Asynchronous JavaScript

Q41. What is the event loop? ★ Most asked

JavaScript is single-threaded, so it runs one thing at a time on the **call stack**. Slow tasks like timers and network calls are handed to the browser/Node APIs. When they finish, their callbacks go into a queue. The **event loop** keeps checking: if the call stack is empty, it moves the next callback from the queue to the stack.

There are two queues: the **microtask queue** (promises) runs first, then the **macrotask queue** (setTimeout, setInterval, I/O).

Q42. What is the output of this code? ★ Most asked

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
```

Output: `A D C B`. Synchronous code runs first (A, D). Then microtasks (promise: C). Then macrotasks (setTimeout: B), even with 0 ms delay.

Q43. What is a Promise? ★ Most asked

A Promise is an object that represents a value that will be available in the future. It has three states: **pending**, **fulfilled** (resolved) and **rejected**.

```
const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve("data"), 1000);
});
p.then(res => console.log(res)).catch(err => console.log(err));
```

Q44. What is async/await? ★ Most asked

`async`/`await` is a cleaner way to write promise code so it looks synchronous. An `async` function always returns a promise. `await` pauses the function until the promise settles.

```
async function getUser() {
  try {
    const res = await fetch("/api/user");
    const data = await res.json();
    return data;
```

```
  } catch (err) {
    console.log(err);
  }
}
```

Q45. What is callback hell and how do you avoid it?

Callback hell is deeply nested callbacks that are hard to read and debug (the "pyramid of doom"). Avoid it with promises, async/await, and by splitting code into small named functions.

Q46. What is the difference between `Promise.all`, `Promise.allSettled`, `Promise.race` and `Promise.any`? ★ Most asked

- `all`: waits for all; fails fast if any one rejects.
- `allSettled`: waits for all and gives every result, success or failure.
- `race`: settles with the first promise that settles (success or failure).
- `any`: resolves with the first success; rejects only if all fail.

Q47. What is the difference between `setTimeout` and `setInterval`?

`setTimeout` runs a function once after a delay. `setInterval` runs it again and again at a fixed interval until you call `clearInterval`.

Q48. How does `fetch` work?

`fetch(url)` makes an HTTP request and returns a promise of a Response. You then call `res.json()` (also a promise) to read the body. Note: `fetch` does not reject on 404 or 500; check `res.ok`.

DOM and browser

Q49. What is the DOM? ★ Most asked

The Document Object Model is a tree of objects that the browser creates from your HTML. JavaScript uses it to read and change the page, e.g. `document.getElementById("title").textContent = "Hi"`.

Q50. What is event bubbling and event capturing? ★ Most asked

When you click an element, the event travels in two phases. **Capturing** goes from the top (window) down to the target. **Bubbling** goes from the target back up to the top. By default, listeners run in the bubbling phase. `event.stopPropagation()` stops it.

Q51. What is event delegation? ★ Most asked

Instead of adding a listener to every child, you add one listener to the parent and check `event.target`. It works because of bubbling. It saves memory and works for elements added later.

```
list.addEventListener("click", e => {
  if (e.target.tagName === "LI") console.log(e.target.textContent);
});
```

Q52. What is the difference between `localStorage`, `sessionStorage` and cookies?

- `localStorage`: about 5-10 MB, stays until cleared.
- `sessionStorage`: same size, cleared when the tab closes.
- Cookies: about 4 KB, sent to the server with every request, can have an expiry, used for sessions/auth.

Q53. What is debouncing and throttling? ★ Most asked

Both limit how often a function runs.

- **Debounce**: run only after the user stops doing something for X ms (search box).
- **Throttle**: run at most once every X ms (scroll, resize).

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
```

Q54. What is the difference between `e.preventDefault()` and `e.stopPropagation()`?

`preventDefault` stops the browser's default action (like a form submitting or a link opening). `stopPropagation` stops the event from bubbling up to parent elements.

ES6+ and misc

Q55. What are the main ES6 features? ★ Most asked

`let`/`const`, arrow functions, template literals, destructuring, default/rest/spread, classes, modules (`import`/`export`), promises, `Map`/`Set`, and `for...of`.

Q56. What are template literals?

Strings written with backticks that allow variables and multi-line text: `` `Hello ${name}` ``.

Q57. What is the difference between CommonJS and ES Modules?

CommonJS uses `require()` and `module.exports` and loads synchronously (old Node style). ES Modules use `import`/`export`, are the standard in browsers, and are loaded statically, which allows tree-shaking.

Q58. What is strict mode?

Adding `"use strict"` makes JS catch common mistakes, like using undeclared variables or duplicate parameter names. ES modules and classes are strict by default.

Q59. What is memoization?

Memoization caches the result of a function call so the same input does not need to be computed again. It is built using a closure and an object/Map as the cache.

Q60. How is JavaScript different from Java?

They are unrelated languages. JS is dynamically typed, interpreted/JIT-compiled, and prototype-based. Java is statically typed, compiled to bytecode, and class-based. The name was a marketing decision.